

What it is

A Python project that ingests robot motion telemetry from files under data/, normalizes it into a single time-series table, runs NumPy-vectorized heuristics (detectors) to flag interesting time windows, compares reported vs motion-derived speed when that comparison is meaningful, and exposes results via python src/analyzer.py (artifacts) and streamlit run app.py (dashboard). Scope is heuristic triage, not certified safety.

End-to-end flow

- 1) Ingest — load_telemetry() (src/io_data.py via src/parser.py). Order: data/telemetry.csv; ROS 2 bag dirs (metadata.yaml); *.mcap; ROS 1 *.bag; *.db3 or parent with metadata.yaml; else synthetic.
- 2) Canonical table — pandas DataFrame df0. Rows are time-ordered samples; df0.attrs holds metadata.
- 3) add_velocity_consistency(df0) -> df (src/robot_truth.py). Adds speed_d, div_speed, div_flag; copies attrs.
- 4) run_all_detectors(df) -> list[SafetyEvent] (src/safety_categories.py). Full-rate vectorized detectors.
- 5) divergence_segments(df) -> DataFrame divs with t_start, t_end, max_div for contiguous div_flag runs.
- 6a) python src/analyzer.py writes safety_events.csv, summary_report.txt, benchmark JSON/txt, PNG charts.
- 6b) app.py: @st.cache_data on load+detect; sidebar window filters plot inputs; KPIs/table use full mission.

Data types in the core pipeline

Telemetry: pandas.DataFrame (float64 typical). Metadata: df.attrs (dict-like).

Detectors: numpy arrays from t, x, y, vx_r, vy_r, omega, speed_r.

SafetyEvent dataclass: t_start, t_end (float), category (str), severity (int), description (str).

Detector output: list[SafetyEvent]. divs: DataFrame [t_start, t_end, max_div] or empty with same headers.

Timing: dict[str, float] from compute_timing_stats(df) — e.g. n_gaps, median_dt_ms, gap_thresh_ms.

Benchmark JSON: dict (rows, slow_seconds, fast_seconds, speedup_pct_vs_slow, ...).

CSV export: DataFrame from events — t_start, t_end, category, severity, description.

Canonical DataFrame columns

After load: t (seconds from clip start), x, y, vx_r, vy_r, omega, speed_r.

After add_velocity_consistency: speed_d, div_speed, div_flag (bool).

Categories (chart order): sudden_stop, sharp_turn, stall, velocity_spike.

Ingest modes

Wheel/pose/odom: robot-truth chart compares reported vs delta-derived speed; divs shades mismatch spans.

IMU-only (imu_fallback): sensor timing/integrity chart; np.gradient used for consistency on that path.

Visualization

Matplotlib figures (src/visualizer.py); Streamlit st.pyplot(..., clear_figure=True).

Event table: list[dict] -> DataFrame -> st.dataframe (Review level in UI; severity in CSV).

Mental model

files -> load_telemetry() -> df0 + attrs -> add_velocity_consistency -> df

-> run_all_detectors -> list[SafetyEvent] -> divergence_segments -> divs

-> analyzer: CSV / TXT / JSON / PNGs -> app.py: cache + (w_lo,w_hi) plot filter -> Streamlit UI